Exercise 2

Github Link

Part 2.

Load the generated data into PostgreSQL in a new table called `orders`.

```
DROP TABLE IF EXISTS orders;

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    customer_name TEXT,
    product_category TEXT,
    quantity INTEGER,
    price_per_unit DECIMAL,
    order_date DATE,
    country TEXT
);

\COPY orders(customer_name,product_category,quantity,price_per_unit,order_date,country) FROM
'/data/orders_1M.csv' DELIMITER ',' CSV HEADER;
```

Using SQL (list of supported SQL commands), answer the following questions:

A. Which order has the highest `price_per_unit`?

```
SELECT *
FROM
    orders
ORDER BY price_per_unit DESC
LIMIT 1;
```

id	customer_name	product_category	quantity	price_per_unit	order_date	country
841292	Emma Brown	Automotive	3	2000	2024-10-11	Italy

B. What are the top 3 product categories with the highest total quantity sold across all orders?

```
SELECT product_category, SUM(quantity) AS sumquality
FROM orders
GROUP BY product_category
ORDER BY sumquality
LIMIT 3;
```

product_category	sumquality
Health & Beauty	300842
Electronics	300804
Toys	300598

C. What is the total revenue per product category? (Revenue = `price_per_unit * quantity`)

```
SELECT orders.product_category, SUM(orders.quantity*orders.price_per_unit) AS revenue
FROM orders
GROUP BY product_category
ORDER BY revenue DESC;
```

product_category	revenue
Automotive	306589798.86
Electronics	241525009.45
Home & Garden	78023780.09
Sports	61848990.83
Health & Beauty	46599817.89
Office Supplies	38276061.64
Fashion	31566368.22
Toys	23271039.02
Grocery	15268355.66
Books	12731976.04

D. Who are the top 5 customers by total spending?

```
SELECT orders.customer_name, SUM(orders.quantity*orders.price_per_unit) AS spending,
COUNT(*) as order_count
FROM orders
GROUP BY customer_name
ORDER BY spending DESC
LIMIT 5;
```

customer_name	spending	order_count
Carol Taylor	991179.18	1028
Nina Lopez	975444.95	980
Daniel Jackson	959344.48	1033
Carol Lewis	947708.57	943
Daniel Young	946030.14	973

E. Look at the spending totals in D — and at how many orders each of those customers has. What do you notice? Open `ecommerce/dataset_generator.py` and explain *why* the data looks like this.

There are 32 `FIRST_NAMES`
There are 33 `LAST_NAMES`
→ 1056 names in total ($F_n * L_n$) (`random_customer_name()`)

The Generator generates $1\,000\,000$ orders (`NUM_ROWS` , so the average `order_count` is $\frac{1\,000\,000}{1056} = 946.\overline{96}$

```
SELECT AVG(order_count) FROM (
    SELECT orders.customer_name, SUM(orders.quantity*orders.price_per_unit) AS spending,
COUNT(*) as order_count
    FROM orders
    GROUP BY customer_name
)
```

→ output: $946.\overline{96}$

The quantity (`random_order()`) is determined by `random.randint(1, 5)` , aka `SELECT AVG(orders.quantity) FROM orders`
→ AVG = 3

The prices of these items vary by item category (I won't calculate that manually)

```
SELECT AVG(spending) FROM (
    SELECT orders.customer_name, SUM(orders.quantity*orders.price_per_unit) AS spending,
COUNT(*) as order_count
    FROM orders
    GROUP BY customer_name
)
```

→ AVG ≈ 810323 (obv. this is not the exact average, but close enough probably.)

Activity *2.2* — Why Is This Self-Join So Slow?

Users of the system sometimes run naive queries such as:

```
SELECT COUNT(*)
FROM people_big p1
JOIN people_big p2
  ON p1.country = p2.country;
```

On the full 1M-row table this takes more than 10 minutes and slows down the whole database for everyone else. Your job is to find out *why* before proposing a fix.

Step 1 — Measure how it grows.

Create three smaller copies of the table:

```
CREATE TABLE people_50k  AS SELECT * FROM people_big WHERE id ⩽ 50000;
CREATE TABLE people_100k AS SELECT * FROM people_big WHERE id ⩽ 100000;
CREATE TABLE people_200k AS SELECT * FROM people_big WHERE id ⩽ 200000;
```

Run the self-join (with `\timing on`) on each of the three tables and fill in:

rows in table	join result (`COUNT(*)`)	time
50 000	27501822	1.838s
100 000	109946508	5.718s
200 000	439395606	21.863s

(quadratic growth)

When the input doubles, by what factor do the result and the time grow?

when the rows double, then the count (&time) quadruples.

Use this to predict the result size and the runtime on `people_big` (1M rows).

$\frac{1\,000\,000}{200\,000} = 5$ scale
since quadratic → $439395606 * 5^2 =$
$\approx 10\,984\,890\,150$ joins and

$21.863s * 5^2 =$
$\approx 546.575s$ $(9,109583\dot{3}min)$

Step 2 — Does an index help?

Create an index on `country` of `people_100k`,
run `ANALYZE people_100k;`, and repeat the query. Did the time change? Use
`EXPLAIN ANALYZE` to support your answer. Why does (or doesn't) the index help?

```
DROP INDEX IF EXISTS people_index;

CREATE INDEX people_index
    ON people_100k (country);

ANALYSE people_100k;

EXPLAIN ANALYSE
SELECT COUNT(*)
FROM people_100k p1
         JOIN people_100k p2
              ON p1.country = p2.country;
```

Without Index ::

QUERY PLAN
Finalize Aggregate (cost=641640.14..641640.15 rows=1 width=8) (actual time=*4189.128*..4189.751 rows=1.00 loops=1)
Buffers: shared hit=2048
→ Gather (cost=641639.92..641640.13 rows=2 width=8) (actual time=4189.114..4189.738 rows=2.00 loops=1)
Workers Planned: 1
Workers Launched: 1
Buffers: shared hit=2048
→ Partial Aggregate (cost=640639.92..640639.93 rows=1 width=8) (actual time=4179.868..4179.870 rows=1.00 loops=2)
Buffers: shared hit=2048
→ Parallel Hash Join (cost=2347.54..478496.13 rows=64857516 width=0) (actual time=125.501..2872.063 rows=54973254.00 loops=2)
Hash Cond: (p1.country = p2.country)

With Index ::

QUERY PLAN
Finalize Aggregate (cost=641316.15..641316.16 rows=1 width=8) (actual time=*3626.924*..3630.131 rows=1.00 loops=1)
Buffers: shared hit=186
→ Gather (cost=641315.94..641316.15 rows=2 width=8) (actual time=3578.623..3630.112 rows=2.00 loops=1)
Workers Planned: 1
Workers Launched: 1
Buffers: shared hit=186
→ Partial Aggregate (cost=640315.94..640315.95 rows=1 width=8) (actual time=3597.673..3597.675 rows=1.00 loops=2)
Buffers: shared hit=186
→ Parallel Hash Join (cost=2332.11..478347.12 rows=64787529 width=0) (actual time=44.177..2299.587 rows=54973254.00 loops=2)
Hash Cond: (p1.country = p2.country)

→ $\frac{4189.128-3626.924}{4189.128} = 13.42\%$ increase
It makes it run faster, but not by much.
This is probably because there are a lot of people in the DB with similar countries.
So every Person has to check all others for matches. (which is very time consuming and isn't really helped that much by having a fast lookup, since it's the number of lookups thats the problem.)
→ there are $10\ 983\ 941\ 260$ matches

Step 3 — Rewrite it.

The query only wants the *number* of matching pairs,
not the pairs themselves. If a country has *k* people, how many pairs does it contribute to the join? Write a query that computes the **same number without a join**. Check that it returns exactly the same result as the join on `people_100k`, then run it on `people_big` and compare its runtime with your prediction from Step 1.

```
SELECT SUM(peoples^2) AS pairs --(This also incl. self joins since the original querry also did that.)
FROM (
    SELECT country, COUNT(*) AS peoples -- How many people belong to each country
    FROM people_big
    GROUP BY country
)
```

→ $109\ 946\ 508 = 109\ 946\ 508$ (people_100k)

Step 4 — Discussion

Considering scalability and efficiency, which approaches and/or optimisations can be applied to improve this kind of query in a real system? Discuss at least:

what you would do if the business actually needed the pairs themselves (not just their count) — would a bigger machine or a cluster help, and how much?

making hardware better (vertical scaling) would help, but since it scales quadratically, it would soon run into the same problems.
Adding additional hardware (horizontal scaling) would possibly be the best course of actions, since this task could be distributed amongst a couple of servers and run in parrallel.
(→ e.g.: one machiene could process AT,DE,FR,... pairs, ...)
It would be slow, but the only way for this to scale at an acceptable level.
(depending on the workload, and how often this needs to be done, there might not be the need to aggressively parallelize, since one server could just spend a couple of days on this task.)

what the rewrite in Step 3 tells you about adding more hardware or an index;

adding an index helps slightly, but not much.
hardware is the same as above.
→ rethinking problems and how to approach them is oftentimes better than "brute-forcing" it.

the limits of an OLTP database for this workload, especially in a large-scale cloud environment.

(depending on the database type) The entire Table could be locked or severely limited until the transaction finishes because of the extreeme overhead or read locks.
→ (posgres uses MVCC to facilitate concurrent writing while data is being read; HOWEVER, mssql doesn't do this, so depending on the type of database, the impact could vary a lot.)

This could make the entire database unusable for it's purpose.
OLTP is also designed for speedy and small transactions, which is the polar opposite of what the join does
→ OLAP would be better suited probably.

the cloud environment will likely make this worse, depending on how it's set up.
On the original Querry, the cloud could unknowingly make gigantic joins without anyone noticing until the bill shoots through a limit.
also, the Speed of transfer would likely become a problem (does data have to be uploaded / pulled from other systems in other availability zones, etc...)
also, a lot of cloud environments do not gurantee that compute will always be available, so that might make querries even slower.